

RAPPORT D'ANALYSE

Projet audité : MYSKY

Critère 1 – Fonctionnalité et respect des User Stories (Note : 16/20)

Le dépôt contient un dossier /Document avec un cahier des charges officiel, des user stories, un MEA/MLD et des maquettes, ce qui permet de vérifier le code par rapport aux engagements pris.

Fonctionnalités effectivement présentes dans le code :

- ❖ Les lecteurs sont identifiés par adresse IP en base (table lecteur), avec découverte automatique via Tailscale (VPN mesh) et vérification de statut UP/DOWN par ping périodique. Une vraie gestion multi-lecteurs réseau existe, pas seulement une démo locale.
- ❖ Transfert de fichiers : pullMP3toPlayers utilise rsync + SSH pour copier les MP3 du serveur vers chaque lecteur, puis déclenche un mpc update.
- ❖ Lecture audio réelle via MPD (Music Player Daemon, piloté en python-mpd2), pas de simulation dans un navigateur.
- ❖ Gestion des urgences/pubs : la méthode Ad() met en pause la lecture en cours sur le lecteur distant, insère la publicité, attend sa durée puis reprend la musique à la position exacte.
- ❖ Une table Log et un service associé permettent de conserver un historique des diffusions.

Lacunes fonctionnelles constatées :

- ❖ La continuité sans réseau (musique locale qui doit continuer en cas de coupure) repose entièrement sur la configuration de MPD *sur la machine lecteur elle-même*. Aucun mécanisme de ce dépôt ne pilote, ne vérifie ni ne teste ce comportement : on ne peut donc pas vérifier qu'il fonctionne réellement, seulement que l'architecture le permettrait en théorie si elle est correctement configurée en dehors du code livré.
- ❖ Aucun bouton d'annulation d'une urgence ou d'une publicité déclenchée par erreur dans l'interface commerciale : une fois lancée, impossible de l'arrêter depuis l'UI.

- ❖ Le tableau de bord admin ne se rafraîchit pas automatiquement : le statut des lecteurs (/get-devices) n'est récupéré qu'au chargement initial de la page, sans aucun polling périodique (setInterval) côté JavaScript. Un lecteur qui tombe en panne ne sera donc visible qu'après un rechargement manuel de la page par l'administrateur.

Critère 2 – Qualité du Code (Note : 13/20)

Architecture MVC avec une séparation claire Controller / Service / DAO / DAOInterface, et un historique Git de 221 commits sur la durée du projet, ce qui montre un usage régulier de l'outil.

Points faibles relevés :

- ❖ 149 occurrences de print() de débogage encore présentes dans le code final (print("hello"), print(f" Données reçues:"), etc.), jamais nettoyées avant le rendu.
- ❖ Chemins absolus codés en dur, propres à la machine de développement : par exemple
/home/servermysky/MYSKY_SAE/PagesCodes/Sae_V2/app/static/audio
dans pullMP3toPlayers. Le code ne fonctionnera donc pas sur une autre machine sans modification manuelle du code source , un paramètre qui aurait dû être externalisé dans un fichier de configuration.
- ❖ Quasiment aucune documentation technique des méthodes complexes : les docstrings, quand elles existent, se limitent à un résumé d'une ligne.
- ❖ Aucun test automatisé n'a été trouvé dans le dépôt.

Critère 3 – Gestion des Données (Note : 14/20)

- ❖ La base de données (musicapp.db) est stockée à la racine de l'application (app.root_path), pas dans un dossier public type /static donc pas de faille d'accès direct par URL.
- ❖ Les mots de passe sont hashés avec bcrypt.
- ❖ Le modèle de données est structuré et normalisé : Groupe_Role, Utilisateur, Fichier_audio, playlist, lecteur, localisation, Log, avec des tables de relation (Ajoute, fait_partie_de, joue_dans) et des clés étrangères correctement déclarées.
- ❖ Le rapport de l'équipe mentionne une protection contre les attaques par traversée de répertoire ("path traversal") et une limite de taille de fichier à l'upload ; le code utilise effectivement secure_filename() côté upload.

Faillles de sécurité concrètes relevées dans le code :

- ❖ Identifiants administrateur par défaut affichés en clair dans le README public du dépôt (leadadmin@mysky.com / myskey123) → visibles par n'importe qui sur GitHub.
- ❖ Clé secrète Flask codée en dur dans le code source versionné (app.secret_key = 'ma cle secrete unique'), ce qui annule une partie de la protection des sessions si le dépôt est public.
- ❖ app.run(debug=True) est laissé actif dans main.py. En environnement de production, le mode debug de Flask est une faille connue (exécution de code arbitraire via la console interactive Werkzeug si elle est exposée sur le réseau).
- ❖ Plusieurs appels subprocess exécutent des commandes avec sudo (sudo tailscale up --reset, sudo rsync ...) directement depuis le processus de l'application web. Cela suppose des droits sudo sans mot de passe configurés pour le compte qui exécute Flask — si ce compte ou le serveur web est compromis, l'attaquant obtient potentiellement des privilèges root.

Critère 4 — Interface Utilisateur (Note : 14/20)

- ❖ L'interface admin permet de gérer les utilisateurs, les rôles, la synchronisation des lecteurs et la consultation des logs avec filtre par date.
- ❖ Le service commercial dispose d'une page de diffusion d'annonces avec historique (nom, durée, statut).
- ❖ L'application se lance en une seule commande (python main.py), avec création automatique de la BDD au démarrage.

Problèmes de confort/supervision constatés :

- ❖ Pas de temps réel : comme noté au critère 1, aucune actualisation automatique du tableau de bord administrateur. Un changement de statut de lecteur n'apparaît qu'après rechargement manuel de la page, ce qui est problématique pour un outil de "supervision".
- ❖ Aucune alerte automatique (sonore, visuelle, mail ou notification) en cas de panne d'un lecteur — uniquement un statut affiché passivement sur la page, qu'il faut donc surveiller activement.
- ❖ Pas de bouton d'arrêt/annulation pour une urgence ou une publicité lancée par erreur.
- ❖ La page commerciale se recharge intégralement toutes les 10 minutes (setInterval + window.location.reload()) plutôt que de rafraîchir les données en arrière-plan, ce qui interrompt l'expérience utilisateur (perte de scroll, de saisie en cours, etc.) à intervalle régulier.

Critère 5 — Installation & Déploiement (Note : 14/20)

- ❖ README clair, dépendances listées dans requirement.txt, lancement en une seule commande, base de données auto-crée au démarrage.
- ❖ La stack technique (Tailscale, MPD, rsync, SSH) est cohérente avec un environnement Linux.

Réserve majeure :

- ❖ Le déploiement réel dépend d'une infrastructure externe qui n'est ni documentée ni automatisée par le dépôt : un réseau Tailscale déjà configuré et authentifié sur chaque machine, des clés SSH de confiance déjà échangées entre le serveur et les lecteurs, et des droits sudo sans mot de passe déjà accordés sur chaque machine. Rien dans le dépôt ne permet de mettre en place cette infrastructure depuis zéro (pas de script de provisioning, pas de documentation de configuration côté lecteur). Sur une machine neuve, le projet ne tournera pas tel quel malgré un README qui laisse penser à un lancement simple en une commande.

CONCLUSION

MYSKY_SAE obtient une note globale de 71/100.

Le projet a une architecture fonctionnelle réelle (réseau via Tailscale, lecture audio via MPD, transfert de fichiers via rsync, base de données structurée) et une certaine rigueur méthodologique au départ (cahier des charges, user stories, MEA). Mais plusieurs problèmes structurels justifient de ne pas le reprendre en l'état :

1. Risques de sécurité concrets et non négligeables : clé secrète et identifiants admin en clair dans le dépôt, mode debug=True actif, et surtout une dépendance à des droits sudo sans mot de passe pour le compte applicatif — un choix d'architecture qui expose le serveur à une élévation de privilèges en cas de compromission.
2. Déploiement non autonome : la totalité de la valeur "réseau" du projet repose sur une configuration externe (Tailscale, SSH, sudo) qui n'est ni documentée ni reproductible à partir du dépôt seul. Reprendre ce projet impliquerait de reconstituer cette infrastructure à l'aveugle.
3. Absence de supervision en temps réel et d'alerting automatique, alors que c'est précisément le cœur d'un outil de supervision : aucune notification de panne, aucun rafraîchissement automatique du tableau de bord, aucune possibilité d'interrompre une diffusion lancée par erreur.

4. Chemins absolus codés en dur propres à la machine de développement d'origine, qui empêcheraient une reprise directe sans réécriture du code.

Décision finale :

notre groupe a décidé de ne pas reprendre le projet réalisé par l'équipe MYSKY, principalement en raison des risques de sécurité identifiés et de la dépendance à une infrastructure réseau non documentée et non reproductible à partir du dépôt.